

庐山派 Lite K230D — 多功能桌面系统

基于 **立创·庐山派-K230-CanMV** 开发板的综合桌面应用程序，集成启动动画、主界面图标启动器、音乐播放器、相机、录音机、相册、画板、WiFi 配网、天气、小说阅读器、待机屏保、系统设置等功能。

一、硬件平台

项目	参数
主控	K230D (CanMV K230)
固件	CanMV_K230D_Lushanpi_Lite_micropython_v1.6-50
屏幕	ST7701 3.1寸 LCD, 800×480, 触摸
摄像头	CSI-2 (sensor_id=2)
蜂鸣器	GPIO61, PWM1, 4000Hz
功放	GPIO10, 高电平使能
触摸	I2C TOUCH(0), 最多 5 点

二、项目架构

```
main.py          # 入口：启动画面 → 主界面
├── setup.py      # 开机启动画面（图片+音频，可跳过）
├── test_touch.py # 主界面：图标网格 + 触摸路由
│   ├── music_player_copy.py # 音乐播放器（WAV播放/暂停/切歌/进度/音量）
│   ├── takephoto.py        # 相机拍照（实时预览/拍照/蜂鸣器提示）
│   ├── audiowr.py          # 录音机（录音/停止/播放/文件列表）
│   ├── picWarehouse.py     # 相册（照片列表/全屏查看/翻页）
│   ├── painting.py         # 画板（触摸绘画/随机颜色/保存/清除）
│   ├── mywificon.py        # WiFi配网（热点+网页配网/状态图标）
│   ├── weather_my.py       # 天气（城市天气/温度/湿度/风向/空气质量）
│   ├── clock_time.py       # 时间同步（NTP + 右上角HH:MM显示）
│   ├── stby.py             # 待机屏保（超时自动进入/点击唤醒）
│   ├── sys_setting.py      # 系统设置（待机超时时间调整）
│   └── novel_reader.py     # 小说阅读器（TXT阅读/进度记忆）
├── ntptime.py        # NTP时间同步模块
└── urequest.py       # HTTP请求模块
```

三、主界面图标布局（800×480）

主界面背景图 /data/code/photo/img_top.jpg，图标区域如下：

第一行 (Y: 90-240) :

相机	音乐播放	WiFi配网	相册	天气预报	
42-160	193-311	344-462	495-612	645-763	

第二行 (Y: 267-417) :

小说阅读	录音机	画板	云平台	系统设置	
42-160	193-311	344-462	495-612	645-763	

右上角 OSD1 层叠加: [HH:MM] [WiFi图标] (绿色已连接 / 红色未连接)

四、各功能模块详解

4.1 开机启动画面 (setup.py)

- 显示 `/data/code/setup/setup.png` + 播放 `/data/code/setup/setup.wav`
- 单击屏幕即可跳过 → 直接进入主界面
- 音频播放完毕自动进入主界面

4.2 音乐播放器 (music_player_copy.py)

- 扫描 `/data/code/music/*.wav` 文件
- 黑胶唱片旋转动画 + 均衡器跳动 + 频谱光环 + 浮动音符
- 触摸控制: 播放/暂停、上一首/下一首、进度条拖拽、音量调节、切歌
- 左侧歌曲列表点击切歌, 深色/浅色主题切换
- 退出按钮返回主界面

4.3 相机 (takephoto.py)

- 实时摄像头预览 (800×480 RGB565)
- 中央白色圆形拍照按钮, 蜂鸣器响一声确认
- 照片保存到 `/data/code/picture/photo_00001.jpg` 自动编号
- 右上角退出按钮, 左上角已拍计数

4.4 录音机 (audiowr.py)

- 录音最长 30 秒, 44100Hz 单声道 WAV
- 录音/停止/播放按钮, 文件列表带时长和播放箭头
- 录音中退出自动保存文件
- 存储目录 `/data/code/audio/rec_001.wav`

4.5 相册 (picWarehouse.py)

- 浏览 `/data/code/picture/` 下的所有 JPG/PNG 照片
- 5行列表, 每行含照片图标、序号、文件名、"查看 >"
- 点击 → 全屏查看 (双图层: 照片+UI叠加)
- 全屏模式: ◀ 上一张 / 下一张 ▶ 按钮 + 退出全屏

4.6 画板 (painting.py)

- 触摸绘画, 手指滑动绘制圆点 (插值平滑)
- 四角大按钮: 左上随机颜色、右上退出、左下清除、右下保存
- 保存到 `/data/code/picture/painting_00001.jpg`, 蜂鸣器确认

4.7 WiFi 配网 (mywificon.py)

- **手机式配网**: 开启热点 `LushanPi-Config` (密码 12345678)
- 手机连接热点 → 浏览器访问 `192.168.4.1` → 选择WiFi输入密码 → K230连接
- 连接成功后自动关闭热点
- 主界面右上角 WiFi 状态图标 (绿色已连接 / 红色未连接)

4.8 天气 (weather_my.py)

- 城市: 临清 (101121707), 数据源: weather.com.cn
- 显示: 城市名、90px温度大字、空气质量色块、湿度、风向、生活建议
- 30秒无操作后进入待机屏保

4.9 待机屏保 (stby.py)

- 主界面超时无操作自动进入 (默认 30 秒, 可在系统设置调整 10~60s)
- 居中显示: 城市名、空气质量、天气信息、日期、时间 (HH:MM)
- 浅色主题, 点击屏幕唤醒返回主界面
- 天气数据每 1000 秒自动更新

4.10 系统设置 (sys_setting.py)

- 调整待机超时时间: 10~60 秒, 步长 5 秒
- - / + 按钮, 当前值绿色显示
- 退出时自动保存

4.11 小说阅读器 (novel_reader.py)

- 扫描 `/data/code/doc/*.txt`, 平铺网格显示 (2×3 卡片)
- 点击卡片 → 显示加载中 → 进入阅读模式
- 阅读界面: 左侧 ◀ 上一页 / 右侧 ▶ 下一页 / 中间区域下一页
- 浅色主题, 顶部返回列表按钮, 底部进度百分比
- **进度记忆**: 自动保存到 `/data/code/doc/doc_read.txt`, 重新打开自动跳转

- 支持 UTF-8 和 GBK 编码

4.12 时间同步 (clock_time.py)

- NTP 时间同步 (ntptime.py , UTC+8)
- 主界面右上角显示 HH:MM 格式时间
- WiFi 连接后自动同步, 每 15 秒刷新

五、所需烧录文件清单

烧录到 K230D 的 Python 文件 (共 19 个)

文件	功能	必需
main.py	程序入口	✓
setup.py	开机启动画面	✓
test_touch.py	主界面 + 图标路由	✓
music_player_copy.py	音乐播放器	✓
takephoto.py	相机拍照	✓
audiowr.py	录音机	✓
picWarehouse.py	相册	✓
painting.py	画板	✓
mywifiicon.py	WiFi配网	✓
weather_my.py	天气	✓
clock_time.py	NTP时间+显示	✓
stby.py	待机屏保	✓
sys_setting.py	系统设置	✓
novel_reader.py	小说阅读器	✓
ntptime.py	NTP时间协议	✓
urequest.py	HTTP请求	✓
old_music.py	旧版播放器 (未启用)	X

```
/data/
├── code/
│   ├── photo/
│   │   └── img_top.jpg          # 主界面背景图（800×480）
│   ├── music/
│   │   └── *.wav                # WAV音乐文件
│   ├── picture/
│   │   ├── photo_*.jpg         # 相机拍摄的照片
│   │   └── painting_*.jpg      # 画板保存的图片
│   ├── audio/
│   │   └── rec_*.wav           # 录音机录制的音频
│   ├── doc/
│   │   ├── *.txt               # 小说TXT文件
│   │   └── doc_read.txt        # 阅读进度（自动生成）
│   ├── video/
│   │   └── new.mp4             # 测试视频
│   ├── setup/
│   │   ├── setup.png           # 启动画面图片
│   │   └── setup.wav           # 启动画面音频
│   └── font/                   # 字体文件（可选）
```

六、注意事项

6.1 硬件注意事项

1. **功放引脚冲突**：功放使用 GPIO10，蜂鸣器使用 GPIO61。FPIOA() 在多处初始化，已做单例保护，不可随意删除 _fpioa_done 判断
2. **触摸单例**：TOUCH(0) 全局只创建一次，test_touch.py 和子应用共享同一实例，重复创建会导致 I2C 总线冲突
3. **Display 初始化顺序**：Display.init() → MediaManager.init()；释放时 Display.deinit() → 延时 60ms → MediaManager.deinit()
4. **应用切换延时**：deinit_home() 中 Display.deinit 和 MediaManager.deinit 之间需 80ms+50ms 延时，否则下一应用的 Display.init 会报 config layer attr failed

6.2 开发注意事项

1. **MicroPython 限制**：
 - 不支持嵌套函数闭包（如 def draw_circle_btn 嵌套在 draw_ui 内会导致 local variable referenced before assignment）
 - 不支持 f-string（用 % 格式化替代）
 - ticks_diff(a, b) 参数顺序是 a - b，不是 b - a
 - PyAudio 在该固件版本无 initialize() 方法
2. **WiFi 配网**：
 - SSID 从 sta.scan() 返回的是 bytes 类型，需 .decode() 转为字符串
 - sta.config('ssid') 返回值可能是 bytes/int，需统一转 str()
3. **图片显示**：

- `Display.show_image(photo.to_rgb565())` 是可靠的图片显示方式
- `draw_image` 配合压缩格式 (JPEG/PNG) 不可靠, 需先 `to_rgb565()`
- 双图层 (OSD0 + OSD1) 可用于照片+UI叠加
- `image.ARGB8888` canvas 的 `clear()` 会使底色透明

4. 音频播放：

- 每帧写入 3-4 个音频块可防止 DMA 缓冲区欠载 (卡顿)
- 不要在触摸回调中直接调用 `stop_stream()` + `close()`, 设标志在 `play_update()` 中清理

5. 内存管理：

- LVGL 双缓冲 (800×480×4×2 ≈ 3MB) 会耗尽可用内存, 分配后 `os.listdir()` 可能卡死
- 文件扫描/大量图片预加载放在 LVGL/大缓冲区分配之前

6. 触摸消抖：

- 启动时需 300-500ms 消抖, 防止主页点击残留触发子应用
- 模式切换 (列表→全屏等) 时重置触摸状态

6.3 待机流程

主界面 → 30秒无操作 → `deinit_home()` → `stby.main()` → 点击唤醒 → `init_home()` → `show_image()`

- 待机超时可在「系统设置」中调整 (10~60 秒)
- 从任何子应用返回主界面后, 倒计时重置

七、快速开始

1. 将上述所有 `.py` 文件烧录到 K230D 的 `/sdcard/` 目录
2. 将资源文件按目录结构放入 `/data/` 路径
3. 放入至少一个 `.wav` 音乐文件到 `/data/code/music/`
4. 上电自动运行 `main.py` → 启动画面 → 主界面
5. 点击图标进入各功能, 点击退出/返回按钮回主界面

八、引脚占用表

引脚	功能	说明
GPIO10	音频功放使能	高电平开启, 退出应用时拉低
GPIO61	蜂鸣器 PWM1	拍照/保存时短鸣 80ms
I2C	触摸屏 TOUCH(0)	全局单例
CSI-2	摄像头 sensor_id=2	拍照/预览
DSI	ST7701 LCD	800×480@30fps

九、全部代码详解

9.1 main.py — 程序入口

```
import setup
import test_touch

if __name__ == "__main__":
    setup.main()          # ① 先运行启动画面
    test_touch.touch_main() # ② 再进入主界面
```

两行代码串联整个项目：启动画面播放完毕后，`setup.main()` 返回，然后进入 `test_touch.touch_main()` 显示主界面。启动画面中已使能功放并初始化 FPIOA，主界面在 `init_home()` 中重新初始化 Display 和 MediaManager。

9.2 setup.py — 开机启动画面

功放初始化（单例模式）：

```
_amp_initied = False
def init_amp():
    global _amp_initied, HT_CTRL
    if not _amp_initied:          # FPIOA 只执行一次
        fpioa = FPIOA()
        fpioa.set_function(AMP_PIN, FPIOA.GPIO10)
        _amp_initied = True
    HT_CTRL = Pin(AMP_PIN, Pin.OUT, pull=Pin.PULL_NONE, drive=7)
    HT_CTRL.high()               # 拉高使能功放
```

K230 的 `FPIOA()` 是全局引脚功能分配器，多次创建会重置所有引脚配置。这里用 `_amp_initied` 标志确保只在首次调用时创建，后续只重新创建 Pin 对象。

跳过检测（机械按键式消抖）：

```
_tap_times = []
_last_press_time = 0
_touch_was_pressed = False
DEBOUNCE_MS = 150

def check_skip(tp):
    now = time.ticks_ms()
    _tap_times = [t for t in _tap_times
                  if time.ticks_diff(now, t) < TAP_WINDOW_MS] # 清理过期记录
    points = tp.read(5)
    if points:
        if not _touch_was_pressed:          # 上升沿检测
            if time.ticks_diff(now, _last_press_time) > DEBOUNCE_MS: # 消抖
                _tap_times.append(now)
                _touch_was_pressed = True
    else:
        _touch_was_pressed = False
    return len(_tap_times) >= TAP_COUNT # 达到点击次数则跳过
```

这是机械按键消抖的经典实现。`_touch_was_pressed` 跟踪触摸状态实现上升沿检测（只在按下瞬间计数，按住不动不重复计数）。`DEBOUNCE_MS=150` 确保两次点击至少间隔 150ms，过滤触摸抖动。

`TAP_WINDOW_MS=2000` 是时间窗口，超出 2 秒的旧记录被清理。

非阻塞音频播放（可中断）：

```
def play_wav_skippable(filepath, tp):
    wf = wave.open(filepath, 'rb')
    chunk = int( framerate / 25 )          # 每块 40ms 音频
    p = PyAudio()
    stream = p.open(output=True, frames_per_buffer=chunk)
    stream.volume(vol=100)

    data = wf.read_frames(chunk)
    while data:
        for _ in range(2):                 # 每次写 2 块
            if data:
                stream.write(data)
                data = wf.read_frames(chunk)
        if check_skip(tp):                 # 每 2 块检查一次跳过
            break
```

关键设计：不是一次性播放完整个文件，而是分块写入，每写入 2 个音频块（约 80ms）就检查一次触摸。这样音频播放和触摸检测可以“并发”执行——用户随时点击屏幕就能中断音频。

主流程：

```
def main():
    init_amp()                                # 使能功放
    Display.init(Display.ST7701, ...)         # 初始化显示屏
    MediaManager.init()                      # 初始化媒体管线
    tp = TOUCH(0)                            # 初始化触摸
    show_setup_image()                       # 显示图片
    play_wav_skippable(SETUP_WAV, tp)        # 播放音频（可跳过）
    # ... finally: Display.deinit() → MediaManager.deinit() → deinit_amp()
```

启动画面退出前必须完整清理硬件资源，因为接下来 `main.py` 会立即调用

`test_touch.touch_main()`，它会在 `init_home()` 中重新初始化 `Display` 和 `MediaManager`。

9.3 test_touch.py — 主界面

图标区域定义：

```
ICON_AREAS = (
    ("相机图标", 42, 90, 160, 240),
    ("音乐播放器图标", 193, 90, 311, 240),
    ("WiFi配网图标", 344, 90, 462, 240),
    ("相册图标", 495, 90, 612, 240),
    ("天气预报图标", 645, 90, 763, 240),
    ("小说阅读器图标", 42, 267, 160, 417),
    ("录音机图标", 193, 267, 311, 417),
    ("画板图标", 344, 267, 462, 417),
    ("云平台监控图标", 495, 267, 612, 417),
    ("系统设置图标", 645, 267, 763, 417),
)
```

每个元组 5 个值：（名称，x1，y1，x2，y2），描述图标在 800×480 背景图上的矩形区域。触摸坐标落在哪个区域就触发对应的功能。

主界面初始化:

```
_shared_tp = None    # 全局触摸单例
def init_home():
    global _shared_tp
    Display.init(Display.ST7701, width=800, height=480, to_id=True)
    MediaManager.init()
    if _shared_tp is None:
        _shared_tp = TOUCH(0)    # 全局只创建一次
    return _shared_tp
```

`_shared_tp` 是触摸设备的全局单例。因为 `TOUCH(0)` 通过 I2C 总线与触控芯片通信，多个实例同时存在会导致总线冲突。所有子应用（音乐播放器、相机等）通过同样的单例模式复用这个触摸实例。

主界面清理（含延时）：

```
def deinit_home():
    Display.deinit()
    time.sleep_ms(80)    # 等显示管线完全释放
    MediaManager.deinit()
    time.sleep_ms(50)    # 等媒体管线完全释放
```

`Display.deinit` 和 `MediaManager.deinit` 之间以及之后的延时是关键。如果立即切换到子应用的 `Display.init()`，会出现 `config layer attr failed` 错误——硬件图层配置还没释放完就被重新申请。

待机倒计时核心逻辑:

```
last_activity = time.ticks_ms()

while True:
    now = time.ticks_ms()
    # 检测超时
    if time.ticks_diff(now, last_activity) > stby.STANDBY_TIMEOUT * 1000:
        deinit_home()
        stby.main()    # 进入待机
        tp = init_home()    # 唤醒后重新初始化主界面
        show_image()
        update_status_bar()
        last_activity = time.ticks_ms()    # 重置计时器

    if points != ():
        last_activity = now    # 有触摸即刷新活动时间
```

`time.ticks_diff(now, last_activity)` 计算 `now - last_activity`，单位为毫秒。当无操作时间超过 `STANDBY_TIMEOUT`（默认 30 秒，可在系统设置中调整），自动进入待机屏保。任何触摸或从子应用返回后，`last_activity` 会被重置。

状态栏更新:

```
def update_status_bar():
    """右上角 OSD1: 时间(HH:MM) + WiFi 图标"""
    clock_time.draw_time_wifi_overlay(mywificon.draw_icon, x=690, y=6)
```

这是主界面右上角的时间和 WiFi 图标组合。通过 OSD1 图层叠加在背景图上，不会影响主界面图标区域的触摸检测。`draw_time_wifi_overlay` 创建一个 100×32 的 ARGB8888 小图像，绘制时间文字和 WiFi 图标后显示在指定坐标。

FPIOA 和 Touch 单例保护：

```

_fpioa_done = False
_shared_tp = None

def init_player_hardware():
    global _fpioa_done, _shared_tp
    if not _fpioa_done:
        fpioa = FPIOA()
        fpioa.set_function(10, FPIOA.GPI010)
        _fpioa_done = True      # FPIOA 只初始化一次
    ...
    if _shared_tp is None:
        _shared_tp = TOUCH(0)  # TOUCH 只创建一次
    tp = _shared_tp

```

这是整个项目中最重要资源管理模式。播放器会在主界面和自身之间反复切换（进入→退出→再进入），每次进入都调用 `init_player_hardware()`。有了这两个单例标志，FPIOA 不会重置其他模块的引脚配置，TOUCH 不会创建重复的 I2C 主控。

音频播放（防欠载）：

```

def audio_play_chunk():
    chunks_per_frame = 3
    for _ in range(chunks_per_frame):
        data = audio_wf.read_frames(audio_chunk_size)
        if data:
            audio_stream.write(data)

```

每帧写入 3 个音频块（不是 1 个）。单个块约 40ms 音频，主循环约 28ms 一帧。如果每帧只写 1 块（40ms 数据），28ms 后 DMA 缓冲区就可能空了——这就是录音机播放卡顿的根因。写 3 块（120ms 数据）能确保 DMA 缓冲区始终有数据。

音频跳转（Seek）：

```

def audio_seek(ratio):
    target = int(audio_total_frames * ratio)
    audio_wf.close()
    audio_wf = wave.open(audio_filepath, 'rb')
    skipped = 0
    while skipped < target:
        audio_wf.read_frames(audio_chunk_size)  # 跳过不需要的帧
        skipped += audio_chunk_size

```

MicroPython 的 `wave` 模块没有 `setpos()` 方法，所以跳转实现为：关闭文件 → 重新打开 → 读取并丢弃目标位置之前的所有帧。对于大文件这会比较慢。

主题切换：

```

def apply_theme(dark=True):
    global C_BG_TOP, C_BG_BOT, C_PANEL, ...  # 所有颜色变量
    if dark:
        C_BG_TOP = (18, 38, 70)  # 深蓝
        ...
    else:
        C_BG_TOP = (238, 242, 248)  # 浅灰
        ...

```

播放器有两套完整的配色方案，通过修改模块级颜色变量来切换。所有绘制函数都引用这些变量，所以修改后下一个渲染帧就会呈现新主题。

9.5 takephoto.py — 相机拍照

摄像头初始化流程：

```
sensor = Sensor(id=2)           # CSI-2 摄像头
sensor.reset()                  # 硬件复位
sensor.set_framesize(width=800, height=480) # 匹配屏幕分辨率
sensor.set_pixformat(Sensor.RGB565) # RGB565 格式
sensor.run()                    # 开始采集
```

摄像头配置为 800×480 RGB565，与 ST7701 屏幕完全匹配，无需缩放。

拍照核心：

```
def save_photo(img, filename):
    data = img.compress(quality=95) # JPEG 压缩
    with open(filename, "wb") as f:
        f.write(data)

def handle_touch(points):
    ...
    if in_rect(x, y, ...): # 点击拍照按钮
        photo_count += 1
        filename = get_next_name() # photo_00001.jpg
        save_photo(img, filename)
        beep() # 蜂鸣器响一声
```

`Sensor.snapshot()` 返回的是 RGB565 像素数据（未压缩）。`img.compress(quality=95)` 将其压缩为 JPEG 字节流，然后写入文件。文件命名自动递增：`photo_00001.jpg` → `photo_00002.jpg` ... 最大 `photo_99999.jpg`。

蜂鸣器（单例）：

```
_fpioa_done = False
def beep_init():
    global _beep_pwm, _fpioa_done
    if not _fpioa_done:
        fpioa = FPIOA()
        fpioa.set_function(61, FPIOA.PWM1) # GPIO61 → PWM1
        _fpioa_done = True
    _beep_pwm = PWM(1)
    _beep_pwm.freq(4000) # 4kHz 频率

def beep():
    _beep_pwm.duty_u16(32768) # 50% 占空比
    time.sleep_ms(80) # 响 80ms
    _beep_pwm.duty_u16(0) # 关闭
```

和功放一样，FPIOA 只初始化一次。PWM 频率 4000Hz 是人耳敏感的中频段，80ms 短促鸣叫不会过于刺耳。

录音引擎 (基于 PyAudio) :

```
def rec_start():
    _rec_chunk = int(44100 / 25)          # 每块 1764 采样点
    _rec_p = PyAudio()
    _rec_stream = _rec_p.open(
        format=paInt16, channels=1, rate=44100,
        input=True, frames_per_buffer=_rec_chunk)

    def rec_update():
        data = _rec_stream.read()          # 读取一个音频块
        _rec_frames.append(data)           # 存入缓冲区
        rec_elapsed_ms = int(_rec_frame_count * _rec_chunk / 44100 * 1000)
        if rec_elapsed_ms >= MAX_REC_SEC * 1000:
            rec_stop()                     # 达到最大时长自动停止
```

录音采用非阻塞架构: `rec_start()` 打开输入流, `rec_update()` 在主循环中每帧读取一个块。这样可以在录音的同时更新 UI (计时器、进度条)。

录音停止与保存:

```
def rec_stop():
    wf = wave.open(filename, 'wb')
    wf.set_channels(1)
    wf.set_sampwidth(_rec_p.get_sample_size(paInt16))
    wf.set framerate(44100)
    wf.write_frames(b''.join(_rec_frames)) # 拼接所有块写入
    wf.close()
    _rec_p.terminate()                     # 释放 PyAudio
```

录音数据以音频块的形式存储在 `_rec_frames` 列表中。停止时用 `b''.join()` 拼接所有块, 一次性写入 WAV 文件。

播放的延迟停止架构:

```
_play_should_stop = False

def play_request_stop():
    """触摸回调中只设标志"""
    _play_should_stop = True

def play_do_stop():
    """在 play_update 中安全关闭"""
    if not _play_should_stop: return
    _play_wf.close()          # 先关文件
    _play_stream.stop_stream() # 停流
    _play_stream.close()       # 关流
    time.sleep_ms(30)          # 等 DMA 释放
    _play_p.terminate()        # 最后关 PyAudio
```

这是解决"点击播放按钮后卡死"问题的关键设计。直接在手势回调中调用 `stop_stream() + close()` 导致音频 DMA 和触摸 I2C 总线死锁。改为延迟架构: 手势回调只设 `_play_should_stop = True`, 实际清理在下一帧的 `play_update()` 中执行。关闭顺序也很重要: 先关 WAV 文件 → 停流 → 关流 → 等 30ms 让 DMA 释放 → 最后 terminate。

9.7 picWarehouse.py — 相册

双图层全屏显示:

```
def draw_fullscreen():
    # OSD0 底层: 照片
    photo = image.Image(filepath)
    Display.show_image(photo.to_rgb565(), layer=Display.LAYER_OSD0)

    # OSD1 顶层: UI 覆盖层
    ui = image.Image(800, 480, image.ARGB8888)
    ui.clear()  # 透明底色
    ui.draw_rectangle(0, 0, 800, 44, color=(0,0,0,180), fill=True)  # 半透明条
    ui.draw_string_advanced(...)  # 文件名、序号、按钮
    Display.show_image(ui, layer=Display.LAYER_OSD1)
```

K230 的 ST7701 显示控制器支持多个 OSD 图层。照片放在 OSD0（底层），UI 元素放在 OSD1（上层）。`ui.clear()` 后 ARGB8888 图像底色透明，只有绘制的矩形和文字可见，其余区域透出底层的照片。注意：OSD1 的触摸坐标和 OSD0 的显示坐标是同一坐标系，按钮位置可直接按 800×480 计算。

模式切换的触摸状态重置:

```
def reset_touch_state():
    global _touch_pressed, _debounce_until
    _touch_pressed = False
    _debounce_until = 0
```

从列表切换到全屏（或反之）时调用，清除前一个模式的触摸残留状态。否则从列表点击进入全屏后，`_touch_pressed` 可能还是 `True`，导致全屏模式的第一次触摸被当作“非首次按下”而忽略。

9.8 painting.py — 画板

绘画插值算法:

```
if last_point is not None:
    dx = x - last_point.x
    dy = y - last_point.y
    dist = (dx*dx + dy*dy) ** 0.5
    if dist <= 30:  # 距离合理才连线
        steps = max(1, int(dist // 8))
        for i in range(1, steps+1):  # 在两点间插值
            nx = int(last_point.x + i * dx / (steps+1))
            ny = int(last_point.y + i * dy / (steps+1))
            canvas.draw_circle(nx, ny, brush_size, ...)
```

触摸屏的采样率有限，手指快速滑动时相邻采样点之间可能间隔几十像素。如果不做插值，画出的线是离散的圆点。插值算法在两点之间按 8 像素步长生成中间点，保证线条连续平滑。`dist > 30` 时认为是新的触摸起点（手指抬起后重新落下），不画连线。

避免 yield 导致的生成器陷阱:

```
def handle_touch(points):
    actions = [] # 用列表收集动作
    ...
    if in_rect(x, y, *CLEAR_BTN):
        return ('clear',) # 返回元组
    ...
    actions.append(('draw', x, y))
    return ('draw', actions) if actions else None
```

早期版本使用 `yield` 返回绘制点，但 `yield` 将函数变为生成器。按钮点击时 `return True` / `return 'save'` 的返回值被 `StopIteration` 吞掉，导致按钮全部失效。改为收集到列表再返回元组，明确区分 `('clear',)`、`('save',)`、`('draw', actions)` 三种返回值。

9.9 mywificon.py — WiFi 配网

热点创建 + HTTP 服务器:

```
def start_ap():
    _ap = network.WLAN(network.AP_IF) # AP 模式
    _ap.active(True)
    _ap.config(ssid="LushanPi-Config", key="12345678")
    _ap_ip = _ap.ifconfig()[0] # 通常 192.168.4.1

def start_server():
    _server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    _server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    _server.bind(('0.0.0.0', 80)) # 监听 80 端口
    _server.listen(1)
    _server.settimeout(0.3) # 非阻塞超时 300ms
```

热点使用 AP 模式创建，手机连接后获得 `192.168.4.x` 地址。HTTP 服务器通过 `settimeout(0.3)` 设为非阻塞模式——主循环每 50ms 调用 `handle_http()`，超时 300ms 内无连接则返回，不阻塞 UI 刷新。

HTTP 请求处理:

```
def handle_http():
    try:
        conn, addr = _server.accept() # 接受连接
        req = conn.recv(1024) # 读取请求
        if b"POST /connect" in req: # 表单提交
            body = req.split(b"\r\n\r\n")[1] # 提取 POST body
            ssid, pwd = parse_form(body) # 解析 SSID 和密码
            conn.send(ok_page) # 返回成功页面
            return True # 通知主循环
        else: # GET /
            conn.send(config_page) # 返回配网页面
    except OSError:
        pass # 超时，无连接
```

配网页面包含一个 `<select>` 下拉列表（由 K230 扫描到的 WiFi 列表填充）和一个密码输入框。用户提交后，K230 解析 POST 请求体，提取 SSID 和密码，然后连接目标 WiFi。

Bytes 类型处理:

```
raw = w.ssid
ssid = raw.decode() if isinstance(raw, bytes) else str(raw)
```

`sta.scan()` 返回的 SSID 是 `bytes` 类型 (如 `b'MyWiFi'`) , 不能直接和字符串拼接。`decode()` 转为 Python 字符串。`sta.config('ssid')` 的返回值也可能是 `bytes`, `is_connected()` 和 `get_connected_info()` 都做了同样处理。

9.10 weather_my.py — 天气

天气数据爬取:

```
def fetch_weather():
    r = urequest.urlopen(WEATHER_URL)          # HTTPS GET
    text = r.read(40000).decode('utf-8')       # 读取网页
    match = re.search(r'var observe24h_data = (.?);', text)
    if match:
        data = json.loads(match.group(1))
        weather[0] = CITY_NAME
        weather[1] = str(data['od']['od2'][0]['od22']) # 温度
        weather[2] = str(data['od']['od2'][0]['od27']) # 湿度
        weather[3] = str(data['od']['od2'][0]['od24']) # 风向
        weather[4] = str(data['od']['od2'][0]['od28']) # 空气质量
```

weather.com.cn 将 24 小时天气数据嵌入在 HTML 的 JavaScript 变量 `observe24h_data` 中。正则表达式提取这个 JSON 对象, 然后解析其中的温度、湿度、风向、空气质量字段。有 3 次重试机制, 失败时保留上次的天气数据。

9.11 stby.py — 待机屏保

时间格式化 (补零):

```
def zero_str(n):
    num = int(n)
    return "0" + str(num) if 0 < num < 10 else str(num)
```

将个位数数字补零显示 (如 `3` → `"03"`) , 用于时间显示保持两位格式。

待机唤醒逻辑:

```
while True:
    points = tp.read(5)          # 检测触摸
    if points:
        if not pressed:
            break                # 任意触摸 → 退出循环 → 返回主界面
        pressed = True
    else:
        pressed = False
    # 绘制待机界面
    img = image.Image(800, 480, image.ARGB8888)
    draw_standby(img)
    Display.show_image(img)
    time.sleep_ms(200)
```

待机界面不断重绘 (200ms 间隔) , 同时检测触摸。按下的瞬间 (`not pressed`) 退出循环, `main()` 返回 → `test_touch.py` 重新初始化主界面。刷新间隔 200ms 足够显示秒级时间变化, 同时 CPU 占用很低。

9.12 sys_setting.py — 系统设置

跨模块共享配置：

```
import stby                                # 导入待机模块

def main():
    timeout = stby.STANDBY_TIMEOUT        # 读取当前配置
    ...
    # 调整
    if 点击减号 and timeout > 10:
        timeout -= 5
    if 点击加号 and timeout < 60:
        timeout += 5
    # 保存
    stby.STANDBY_TIMEOUT = timeout        # 写回模块变量
```

Python 模块是单例的，`stby.STANDBY_TIMEOUT` 在 `stby.py` 中定义为模块级变量。

`sys_setting.py` 通过 `import stby` 访问和修改它。`test_touch.py` 在待机检测中读取 `stby.STANDBY_TIMEOUT` 判断超时时间。这是 MicroPython 中最简单的跨模块状态共享方式——不需要文件或数据库。

待机时间范围是 10~60 秒，步长 5 秒，防止设置过于极端。

9.13 novel_reader.py — 小说阅读器

文本分页算法：

```
CHARS_PER_LINE = 680 // 18 # ≈37 字符/行
LINES_PER_PAGE = 410 // 22 # ≈18 行/页

def split_pages(text):
    lines = []
    for paragraph in text.split('\n'):
        para = paragraph.rstrip()
        while len(para) > 0:
            lines.append(para[:CHARS_PER_LINE]) # 截断超长行
            para = para[CHARS_PER_LINE:]
    # 按每页行数分页
    for i in range(0, len(lines), LINES_PER_PAGE):
        pages.append(lines[i:i+LINES_PER_PAGE])
    return pages
```

以段落为单位处理，每个段落的超长行按 `CHARS_PER_LINE` 截断。然后用 `LINES_PER_PAGE` 将行列表切片为页面。这是最简单的分页方案——不做中文字符宽度计算（等宽字体下每个中文约等于 1 个英文字符宽度），不处理标点避头尾。

多编码兼容：

```
def read_file_text(filepath):
    data = open(filepath, 'rb').read()
    if data[:3] == b'\xef\xbb\xbf': # UTF-8 BOM
        data = data[3:]
    try: return data.decode('utf-8')
    except: pass
    try: return data.decode('gbk') # 中文 Windows 常用编码
    except: pass
    return data.decode('utf-8', 'ignore') # 最终回退
```


先尝试 UTF-8（现代标准），失败后尝试 GBK（中文 Windows 常用编码），最终用 `ignore` 模式强制解码（跳过无法解码的字节）。处理 BOM（字节序标记）防止文件开头的 `\xef\xbb\xbf` 被当作内容。

进度持久化:

```
def save_progress(filename, page, total):
    prog = load_progress()           # 读取所有进度
    prog[filename] = page            # 更新当前文件
    with open(PROGRESS_FILE, 'w') as f:
        for fn, pg in prog.items():
            f.write(fn + "=" + str(pg) + "\n")
```

进度以 `文件名=页码` 的格式存储在 `/data/code/doc/doc_read.txt` 中，每行一个文件。读取时解析为字典，保存时重写整个文件。这是最简单的持久化方案，适合少量文件的场景。

性能优化——按需重绘:

```
last_page = -1
while True:
    ...
    if screen == 'reading':
        if page_changed or last_page != cur_page:
            img = image.Image(...)
            draw_reading(img, cur_novel, pages, cur_page)
            Display.show_image(img)
            last_page = cur_page
        time.sleep_ms(30)           # 仅 30ms 循环
    else:
        ...
        time.sleep_ms(80)          # 网格界面 80ms 循环
```

阅读界面只在翻页时重绘（`last_page != cur_page`），空闲时只做触摸检测 + 30ms 休眠。这比每帧重绘（800×480 ARGB8888 图像创建+18行文字绘制）节省大量 CPU，解决了“小说很卡”的问题。

9.14 clock_time.py — 时间同步

NTP 时间同步:

```
def sync_time():
    import ntptime
    ntptime.settime()           # 同步 RTC
```

`ntptime.py` 使用 UDP 协议向 NTP 服务器发送请求，接收 48 字节的 NTP 响应包，解析出 UTC 时间戳，加上 UTC+8 时区偏移后设置 K230 的 RTC（实时时钟）。之后 `time.localtime()` 就能获取正确时间。

状态栏组合绘制:

```
def draw_time_wifi_overlay(wifi_icon_drawer, x, y):
    img = image.Image(100, 32, image.ARGB8888)
    img.clear()
    img.draw_string_advanced(4, 6, 18, get_time_str(), color=(255,255,255))
    wifi_icon_drawer(img, 68, 5)    # 委托 mywifiicon 绘制 WiFi 图标
    Display.show_image(img, x=x, y=y, layer=Display.LAYER_OSD1)
```

接收 `wifi_icon_drawer` 回调函数作为参数，将 WiFi 图标的绘制委托给 `mywifiicon.draw_icon`。组合后的图像通过 OSD1 图层叠加在主界面右上角，不影响图标区域触摸。

9.15 ntptime.py — NTP 协议实现

```
def time():
    NTP_QUERY = bytearray(48)
    NTP_QUERY[0] = 0x1B # NTP 请求标识
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # UDP
    s.sendto(NTP_QUERY, addr)
    msg = s.recv(48)
    val = struct.unpack("!I", msg[40:44])[0] # 解析 32 位时间戳
    if val < 3913056000: # Y2036 修复
        val += 0x100000000 # 加 2^32
    return val - 2208988800 # NTP → Unix 时间戳
```

NTP 时间戳是 1900 年起的 32 位秒计数，2036 年溢出。代码用 3913056000（2024 年的 NTP 时间戳）作为阈值判断溢出并修正。最后减去 2208988800（1900→1970 的秒数差）转换为 Unix 时间戳。

9.16 urequest.py — HTTP 客户端

```
def urlopen(url, data=None, method="GET"):
    proto, dummy, host, path = url.split("/", 3) # 解析 URL
    port = 80 if proto == "http:" else 443 # 端口
    ai = socket.getaddrinfo(host, port)[0] # DNS 解析
    s = socket.socket(ai[0], ai[1], ai[2]) # TCP 连接
    s.connect(ai[-1])
    if proto == "https:":
        s = ssl.wrap_socket(s) # SSL 加密
    s.write(method + " /" + path + " HTTP/1.0\r\n...\")
    return s # 返回 socket 供 read()
```

一个轻量级 HTTP/HTTPS 客户端。对于 HTTPS URL，通过 `ssl.wrap_socket()` 建立 TLS 加密连接。返回原始 socket 对象，调用者通过 `.read()` 读取响应体。天气模块用它访问 `weather.com.cn` 的 HTTPS 接口。

十、程序启动完整流程

```
上电 → boot.py → main.py
|
├─ 1. setup.main()
|   ├── init_amp()      GPIO10 功放使能
|   ├── Display.init()  ST7701 初始化
|   ├── MediaManager.init()
|   ├── show_setup_image() 显示开机图
|   ├── play_wav_skippable() 播放音频(可跳过)
|   └── Display.deinit() + MediaManager.deinit()
|
├─ 2. test_touch.touch_main()
|   └── main()
|       ├── init_home()      重新初始化显示+触摸
|       ├── show_image()     显示主界面背景
|       ├── update_status_bar() 时间+WiFi图标
|       └── while True:      主事件循环
|           ├── 触摸检测 + 待机超时检测
|           ├── 图标点击 → deinit_home() → 子应用.main()
|           └── 子应用返回 → init_home() → 恢复主界面
```

每个子应用遵循统一的模式：`Display.init()` → `MediaManager.init()` → 主循环 → `Display.deinit()` → `MediaManager.deinit()`。`deinit_home()` 在切换到子应用前清理主界面资源，子应用的 `deinit_*`() 在返回前清理自身资源，`init_home()` 恢复主界面。